4
Amazon Database Services

Amazon database services provide a scalable, flexible, and high-performance solution for a wide range of applications and workloads. These services are fully managed, meaning that AWS handles the underlying infrastructure and administration tasks such as managing high availability and scalability. This allows users to focus on their data and applications. They also integrate with other AWS services, allowing for seamless integration with other parts of an AWS application stack.

In this chapter, we will cover the following Amazon database services:

Amazon cloud relational databases: Fully managed AWS relational databases that can be used for transactional applications, reporting and analytics, and content management, including **Amazon Relational Database Service** (**RDS**) for traditional databases such as MySQL, MariaDB, PostgreSQL, Oracle, SQL Server, and **Amazon Aurora** for a high-performing, MySQL and PostgreSQL-compatible database.

Amazon cloud NoSQL databases: Fully managed AWS NoSQL databases that can handle large volumes of unstructured data with high scalability and availability, including **Amazon DynamoDB**, a key-value and document database, and **Amazon DocumentDB**, a MongoDB-compatible document database.

Amazon cloud in-memory caching: Fully managed AWS in-memory data caching services that can be used to improve application performance by caching frequently accessed data, including **Amazon ElastiCache** for **Redis** and **Memcached**.

Amazon Cloud Data Warehousing: Fully managed data warehousing services that can be used to store and analyze large volumes of structured and unstructured data, including **Redshift** for large-scale data warehousing, and **Amazon EMR** (**Elastic MapReduce**) for processing large datasets using Apache Hadoop, Spark, and other big data tools. We will focus on Redshift in this chapter and discuss the others in the next chapter.

By following these topics in this chapter, you will understand the basic concepts and gain hands-on skills with Amazon cloud database services. If you are familiar with database basics and **structured query language** (**SQL**), a programming language for storing and processing information in a relational database, you can skip the first section. The examples shown in this chapter are also available in the GitHub repository for this book: `https://github.com/PacktPublishing/Self-Taught-Cloud-computing-Engineer`.

Database basics

A **database** is an organized collection of data that is stored and managed on a computer. It allows for efficient storage, retrieval, and management of data, which can be used for a wide range of applications, such as websites, mobile apps, financial systems, and more. A database consists of one or more tables that contain related data. Each table contains rows and columns, where the columns represent the attributes or fields of the data, and the rows represent the individual instances or records of the data. There are many different types of databases:

- **Relational databases**: These are the most common type of database, where structured data is organized in a tabular format with rows and columns. Data is organized into tables, and relationships between tables are established through keys.
- **NoSQL databases**: These databases are designed to handle unstructured or semi-structured data that cannot be easily organized into tables. Instead, they use document-based, key-value, or graph-based approaches to store and organize data.
- **Object-oriented databases**: These databases store data as objects rather than in a tabular format. They are designed to work with object-oriented programming languages, such as Java, C++, and so on.
- **Data warehouses**: These databases are used to store large amounts of data from different sources, such as transactional systems, social media, and other data sources. They are designed to support complex data analysis and reporting

Overall, databases are an essential tool for managing and organizing data and are used in many different industries and applications. They provide a scalable, efficient, and secure way to store and manage data, which can be accessed by multiple users and applications at the same time. We will discuss each of them in the upcoming sections.

Relational databases

A relational database stores and organizes data in tables or relations, with each table representing a specific entity or concept. The tables are made up of columns and rows, where each column represents a particular attribute or characteristic of the entity, and each row represents a specific instance or record of the entity. We use a schema to define the structure of a database, including the tables and relationships between them.

A schema is a map of how data is stored and organized in the database. It has tables and keys including a **primary key** and a **foreign key**. A primary key is a column or group of columns in a table that uniquely identifies each row in that table: no two rows have the same value in the primary key column(s). Primary keys are important for maintaining the integrity and consistency of the data in the database, as they ensure that each record can be uniquely identified and retrieved. A foreign key is a column in one table that refers to the primary key of another table. It establishes a link or relationship between the two tables, enabling data to be retrieved from multiple tables at the same time. Foreign keys are

essential for maintaining referential integrity in a database, which means that data in one table must always correspond to data in another table.

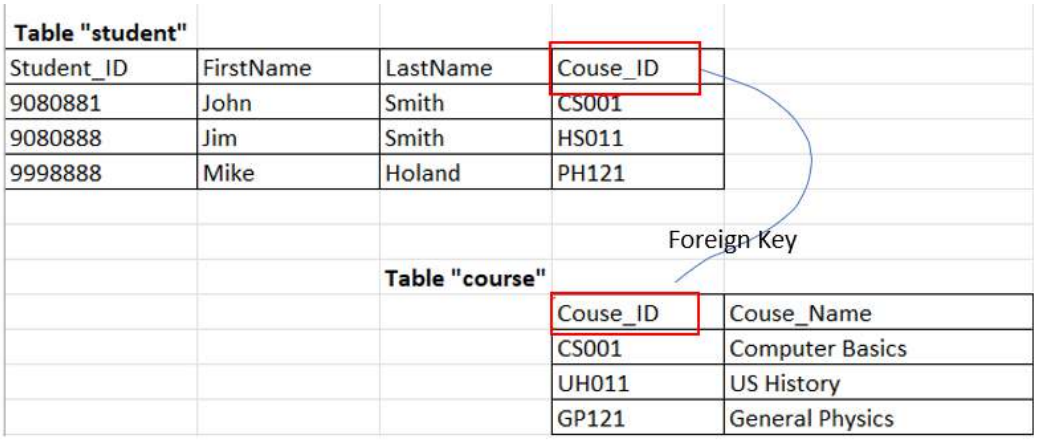


Figure 4.1 – Database schema

Figure 4.1 shows a small database consisting of two tables: a `student` table and a `course` table. The primary key for the `course` table is `Course_ID` ,which is also a foreign key in the `student` table. This relationship defines the database schema and allows you to retrieve information about both students and the courses they take using a single query.

Relational databases use **Structured Query Language** (**SQL**) to create, manipulate, and retrieve data. SQL allows users to write complex queries that can extract specific data from one or more tables, based on various conditions or criteria. The ability to query data in this way makes relational databases a powerful tool for data management and analysis. The following examples create a database called `school`, and create a table called `students` in the database, with `StudentID` as the primary key:

```
Create database school;
Create table school.students (
    StudentID int primary key,
    LastName varchar(100),
    FirstName varchar(100),
    City varchar(100)  );
```

Relational databases have **Atomicity, Consistency, Isolation, and Durability** (**ACID**) to ensure data reliability and transactional integrity in database transactions. More details are available at `https://aws.amazon.com/relational-database/`.

NoSQL databases

NoSQL databases were designed to handle unstructured or semi-structured data and to support highly scalable and distributed systems. Different from relational databases , which rely on structural data models, NoSQL databases use a more flexible, non-relational data model such as key-value pairs, documents, graphs, and other formats. There are several different types of NoSQL databases, including document-oriented databases, key-value stores, graph databases, and column-family stores. Each type has its unique strengths and weaknesses and is best suited for different types of applications and data models.

NoSQL databases are more flexible since there is no schema among the tables. The following example shows a `Student` table that has entries with the key-value format:

```
Student =
{ "Student class number" : 101,
"Student details": [


{"Student name" : "John Smith", "Born City" : "Houston", "Course_
taking" : "CS201"},


{"Student name" : "Jim Smith", "Born City" : "Plano", "Age" : 20},


{"Student name" : "Neil Armstrong", "Age" : 18, "Course_taking" :
"CS202"}


{"Student name" : "Neil Smith", "Age" : 21, "Born City" : "Chicago"}

],}
```

NoSQL databases are not a replacement for traditional relational databases. Rather, they are complementary technology that can be used in conjunction with them to address specific data management needs.

In-memory cache databases

An **in-memory cache database**, also known as an **in-memory data grid**, is a type of database that stores data entirely in RAM for very fast data access and retrieval. In-memory cache databases are often used to accelerate the performance of high-traffic, latency-sensitive applications.

Here are some examples of in-memory cache databases:

- **Redis**: In addition to being a popular NoSQL database, Redis can also be used as an in-memory cache database. It supports a wide range of data structures and can be used for caching, messaging, and more.

- **Memcached**: A high-performance, distributed in-memory cache database that is widely used for caching web applications. It can be used to store key-value pairs and can be accessed from multiple servers in a cluster.

In-memory cache databases are often used in conjunction with other databases, such as a traditional disk-based relational database, to provide a fast and efficient caching layer for frequently accessed data.

Data warehouses

The goal of a **data warehouse** is to provide a comprehensive, unified view of an organization's data so that it can be used for decision-making, reporting, and other analytical purposes. They are also optimized for fast querying and reporting and often use specialized tools and technologies, such as **online analytical processing** (**OLAP**) and data mining, to extract insights and intelligence from data.

Data warehouses are typically used by large organizations that need to analyze and report on large amounts of data from multiple sources. They are commonly used in industries such as finance, retail, healthcare, and telecommunications, where there is a need to analyze data from many different sources and perform complex analyses and reporting. *Figure 4.2* shows examples of data warehouse functions:

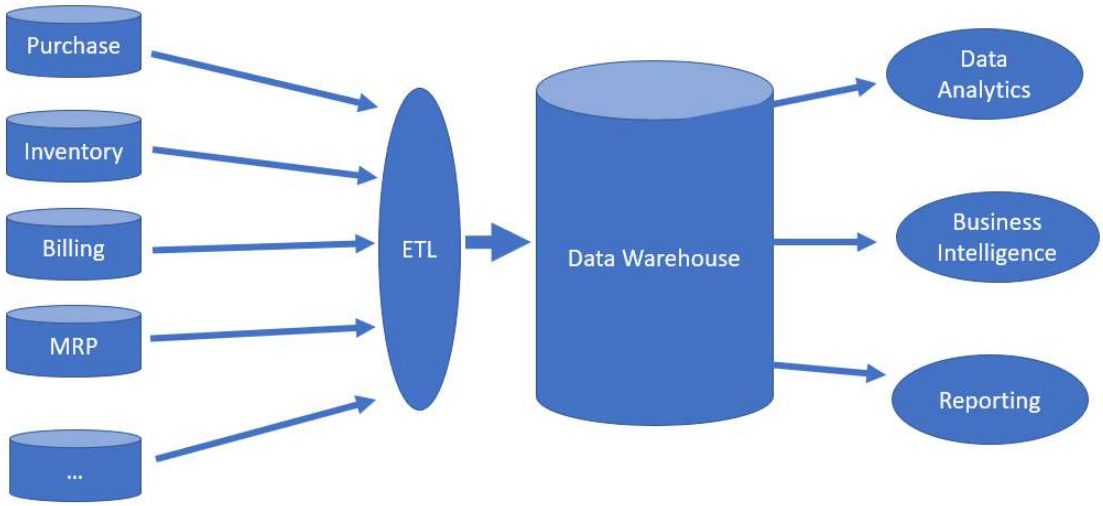


Figure 4.2 – Data warehouse functions

By **Extracting, Transforming, and Loading** (**ETL**) from various sources, data is ingested into the data warehouse, which is a large, centralized data repository, in a structured format that is optimized to conduct data analytics and business intelligence, and generate reporting. After we have briefed the different types of AWS cloud database services, we will now closely examine each of these services.

Amazon RDS

RDS is a fully managed relational database service provided by Amazon in the cloud. With RDS, AWS provides a few popular relational database management systems as a service, including MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB.

Amazon RDS is a managed service. There are different ways to create a database, such as MySQL, in the cloud. One is provisioning an EC2 instance and installing a **database management system** (**DBMS**) on the instance, and then creating databases, tables, and so on. This is an unmanaged approach since you will have to take care of all the administrative tasks such as database scaling, high availability, disaster recovery, and so on. Another is using cloud-managed services to provision a database where AWS will handle the mundane and time-consuming tasks such as hardware provisioning, software installation, backup and recovery, patching, scaling, and so on. Managed services allow users to focus on their applications and data without worrying about the underlying infrastructure.

RDS is a managed service. It provides high availability and fault tolerance by replicating the database to multiple **AZs** and providing automatic failover in the event of a primary database failure. It also supports automated backups and point-in-time restores, ensuring data durability and recoverability. RDS allows users to scale databases vertically as needed with minimal downtime.

As an AWS cloud service, RDS integrates with other AWS services, such as Amazon CloudWatch, which provides monitoring, alarms, and AWS IAM, which allows users to control who can access their databases and what they can do with them.

Recall that an EC2 instance in the public subnet can send or receive traffic directly to and from the internet, whereas an instance in the private subnet can only access the internet via a NAT gateway in the public subnet. This holds true for the database instances in different subnets. A typical multi-tier design is to expose the web tier to the internet while hiding the app tier and database tier in the private subnets. *Figure 4.3* shows an architecture of multi-tiers including web, application, and RDS databases:

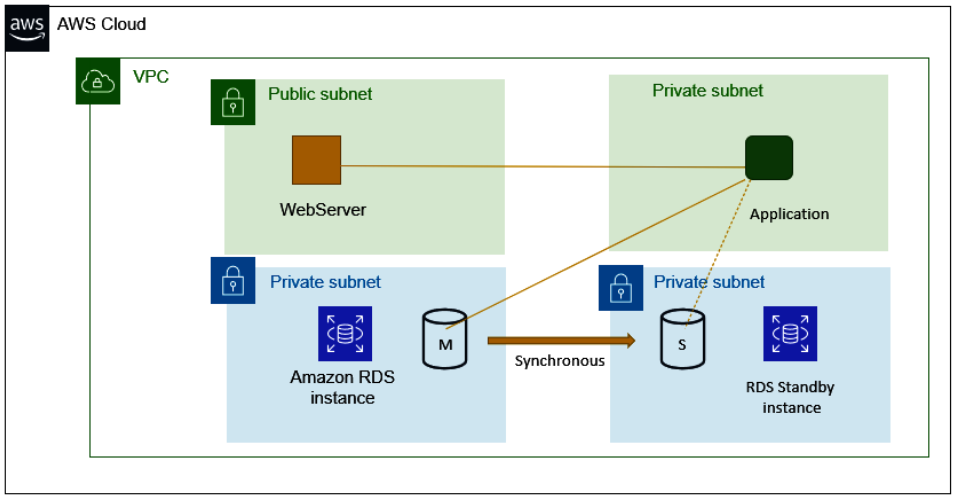


Figure 4.3 – Multi-tier architecture with a multi-AZ RDS

The web server instance is sitting in the public subnet, the application instance sits in the private subnet, and the RDS instance is configured as multi-AZ with a master server in one subnet and the slave server in another so that the slave can take charge when the master is down. We will now implement the RDS portion of the architecture and connect it from an EC2 instance, step by step (for simplicity, we use Zonal RDS):

1. **From the AWS Console | RDS | Create Database**: Fill in the details and click **Create Database**. *Figure 4.4* shows a summary of the database:

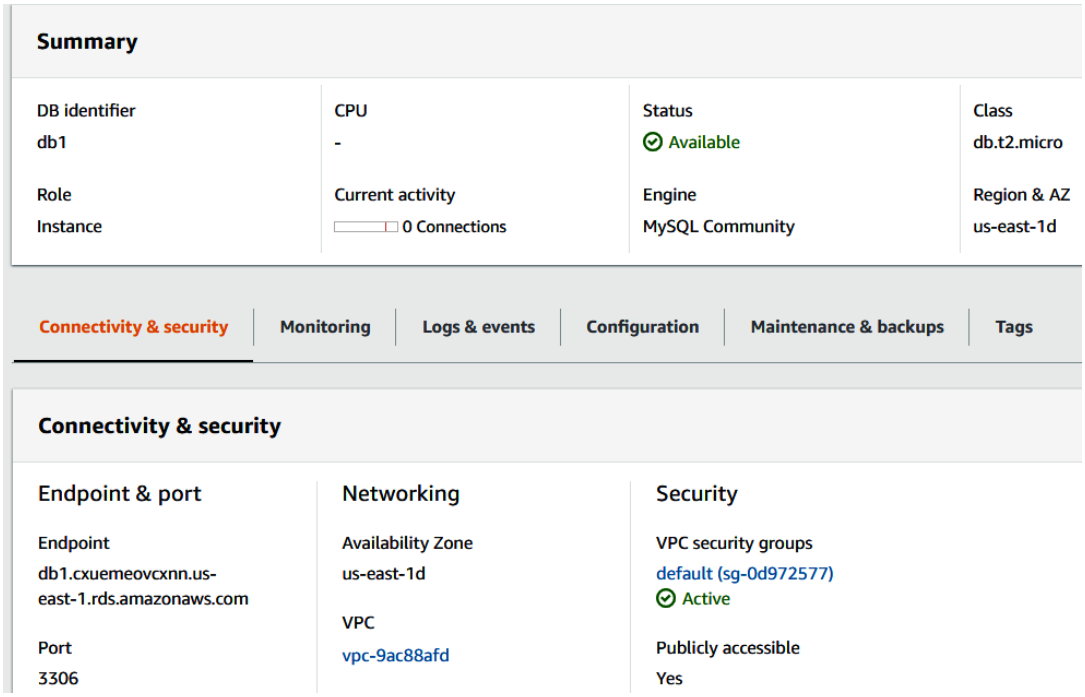


Figure 4.4 – RDS database

2. **Connect from an EC2 instance**: Using `Putty` to connect to a Linux instance, install `mysql client pkg` and connect to the RDS endpoint:

```
sudo yum install mysql
mysql --user admin --password --host db1.cxuemeovcxnn.us-east-1.
rds.amazonaws.com
```

3. Once connected to the RDS instance, we will create a database and a table, insert entries into the table, and run some SQL queries, as shown in *Figure 4.5*:

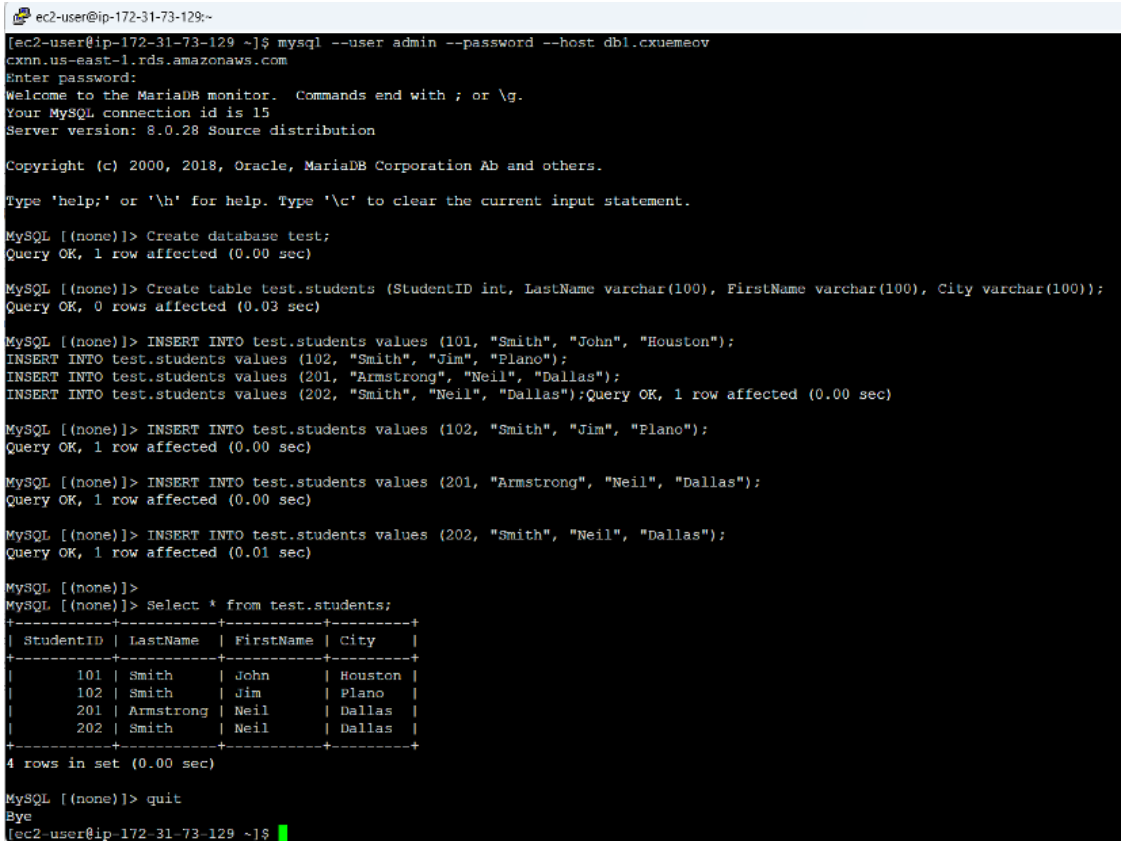


Figure 4.5 – Connect and operate the RDS database

AWS RDS provides an option that enables users to create one or more read-only copies of their database instance within the same AWS region or across different AWS regions, called `read- replicas`. By copying the data from the primary/source database instance to the replica instance, you can offload read traffic from the primary instance to the replica. Because a read replica is read-only, any write operations must still be performed on the primary instance. The read replica can be used to improve the performance of read-intensive workloads or to create a DR solution. In the case of DR, if the primary instance fails, the read replica can be promoted to the primary instance, reducing downtime.

AWS RDS read replicas can be created for various database engines, including MySQL, PostgreSQL, Oracle, and SQL Server. The replica instances can be resized, backed up, and deleted independently of the primary instance. Additionally, users can configure the replication settings, including the replication frequency and the source and destination databases.

In addition to cloud RDS, Amazon offers another powerful cloud relational database service, Amazon Aurora, which is highly available globally, highly scalable, durable, and performs great. Amazon Aurora has many features:

- It uses a distributed architecture that enables its scaling horizontally across multiple nodes, while maintaining low latency and high throughput
- It can automatically replicate data across multiple AZs for high availability and durability
- It supports both across-region read replicas and global databases
- It offers point-in-time recovery, allowing the database to restore to a specific point in time
- It's compatible with MySQL and PostgreSQL – existing applications and tools that use MySQL/PostgreSQL can be migrated to Aurora easily
- It offers advanced security features such as encryption at rest and in transit

Overall, AWS RDS and Aurora are powerful and flexible cloud relational database services that are highly available, scalable, and cost-effective. They are great choices for organizations of all sizes, from start-ups to enterprises that need to manage their relational databases in the cloud. While RDS suits small or medium-sized databases with relatively low cost, Aurora is a global petabyte-scale database for mission-critical applications and is thus more expensive than RDS. More details about AWS relational databases can be found at `https://aws.amazon.com/rds/pricing/` and `https://aws.amazon.com/rds/aurora/pricing/`.

In the next section, we will examine the NoSQL databases in the AWS cloud.

Amazon cloud NoSQL databases

DynamoDB is a fully managed NoSQL database service provided by Amazon in the cloud. It is a key-value and document database that can handle structured, semi-structured, and unstructured data, making it a versatile solution for a wide range of applications. It is also designed to be highly available, with automatic multi-AZ replication for built-in redundancy and durability.

DynamoDB can scale seamlessly with no downtime or disruption, allowing users to scale their databases up or down as needed, with pay-as-you-go pricing based on the amount of data stored and the number of requests made.

DynamoDB is designed for high performance with sub-millisecond latency for read and write operations, making it an ideal choice for real-time, low-latency applications. It is also highly secure, with support for encryption at rest and in transit, as well as fine-grained access control through AWS IAM. We will create a DynamoDB through AWS Cloudshell:

```
aws dynamodb create-table \
--table-name Music \
--attributedefinitions \
AttributeName=Artist, AttributeType=S \
```

```
AttributeName=SongTitle, AttributeType=S \
--key-schema \
AttributeName=Artist,KeyType=HASH \
AttributeName=SongTitle, KeyType=RANGE \
--provisionedthroughput \
ReadCapacityUnits=5,WriteCapacityUnits=5 \
--table-class STANDARD
```

As shown in *Figure 4.6*, the DynamoDB table `Music` is created:

```
[cloudshell-user@ip-10-2-48-206 ~]$ aws dynamodb create-table \
>     --table-name Music \
>     --attribute-definitions \
>         AttributeName=Artist,AttributeType=S \
>         AttributeName=SongTitle,AttributeType=S \
>     --key-schema \
>         AttributeName=Artist,KeyType=HASH \
>         AttributeName=SongTitle,KeyType=RANGE \
>     --provisioned-throughput \
>         ReadCapacityUnits=5,WriteCapacityUnits=5 \
>     --table-class STANDARD
{
    "TableDescription": {
        "AttributeDefinitions": [
            {
                "AttributeName": "Artist",
                "AttributeType": "S"
            },
            {
                "AttributeName": "SongTitle",
                "AttributeType": "S"
            }
        ],
        "TableName": "Music",
        "KeySchema": [
            {
                "AttributeName": "Artist",
                "KeyType": "HASH"
            },
            {
                "AttributeName": "SongTitle",
                "KeyType": "RANGE"

              }
         ],
         "TableStatus": "CREATING",
         "CreationDateTime": "2023-02-21T20:36:32.696000+00:00",
         "ProvisionedThroughput": {
             "NumberOfDecreasesToday": 0,
             "ReadCapacityUnits": 5,
             "WriteCapacityUnits": 5
         },
         "TableSizeBytes": 0,
         "ItemCount": 0,
         "TableArn": "arn:aws:dynamodb:us-east-1:317332158300:table/Music",
         "TableId": "ded6c371-078f-420d-bf03-e604c1459deb",
         "TableClassSummary": {
             "TableClass": "STANDARD"
         }
    }
}
[cloudshell-user@ip-10-2-48-206 ~]$
```

Figure 4.6 – Create a DynamoDB database

In the preceding DynamoDB table creation command, there are some concepts we need to understand:

- **Key schema**: A DynamoDB table has two types of keys:

  The **hash key**, also known as the **partition key**, is a required attribute that uniquely identifies items in a table. The hash key is used to partition the table's items across multiple physical storage partitions for scalability and performance. For example, for the `Music` table, we choose the `Artist` attribute as the hash key to ensure that all items for a given artist are stored in the same partition, which allows for fast access to that artist's data.

  The **range key**, also known as the **sort key**, is an optional attribute that is used in combination with the hash key to uniquely identify items in a table. The range key is used to further sort and organize items within a particular hash key partition. For example, for the `Music` table, we choose the `SongTitle` attribute as the range key to ensure that all items for a given artist are sorted by song titles within their partition.

  The combination of a hash key and a range key is called a **composite primary key**. This allows for efficient querying of items in a table based on both the hash key and range key values.

- **Provisioned throughput**: This is a setting that determines the amount of read and write capacity that is provisioned for a table or an index. Provisioned throughput is specified as two separate values: the **read capacity unit** (**RCU**) and the **write capacity unit** (**WCU**). One RCU represents the capacity to read up to 4 KB of data per second, while one WCU represents the capacity to write up to 1 KB of data per second.

  Provisioned throughput is important because it determines the rate at which your application can read and write data to the table or index. If your application exceeds the provisioned throughput capacity, then DynamoDB may start returning throttling errors. On the other hand, if you provision more capacity than you need, you will be paying more for unused capacity. In the DynamoDB instance created earlier, we have set an RCU of 5 and a WCU of 5, now let's add some items to the table and run some queries against it:

1. **Add items to the DynamoDB table**:

```
aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Broadway"}, "SongTitle": {"S": "Call Me"},
"AlbumTitle": {"S": "Somewhat Famous"}}'

aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Broadway"}, "SongTitle": {"S": "Howdy"},
"AlbumTitle": {"S": "Famous"}, "Awards": {"N": "2"}}'

aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Acme Band"}, "SongTitle": {"S": "Happy"},
"AlbumTitle": {"S": "About Life"}, "Awards": {"N": "10"}}'
```

```
aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Acme Band"}, "SongTitle": {"S": "Rocks"},
"Awards": {"N": "8"}}'
```

2. **Query the table in the AWS console**:

   Going to **AWS Console** | **DynamoDB** | **Tables**, we can see the table shown in *Figure 4.7*:

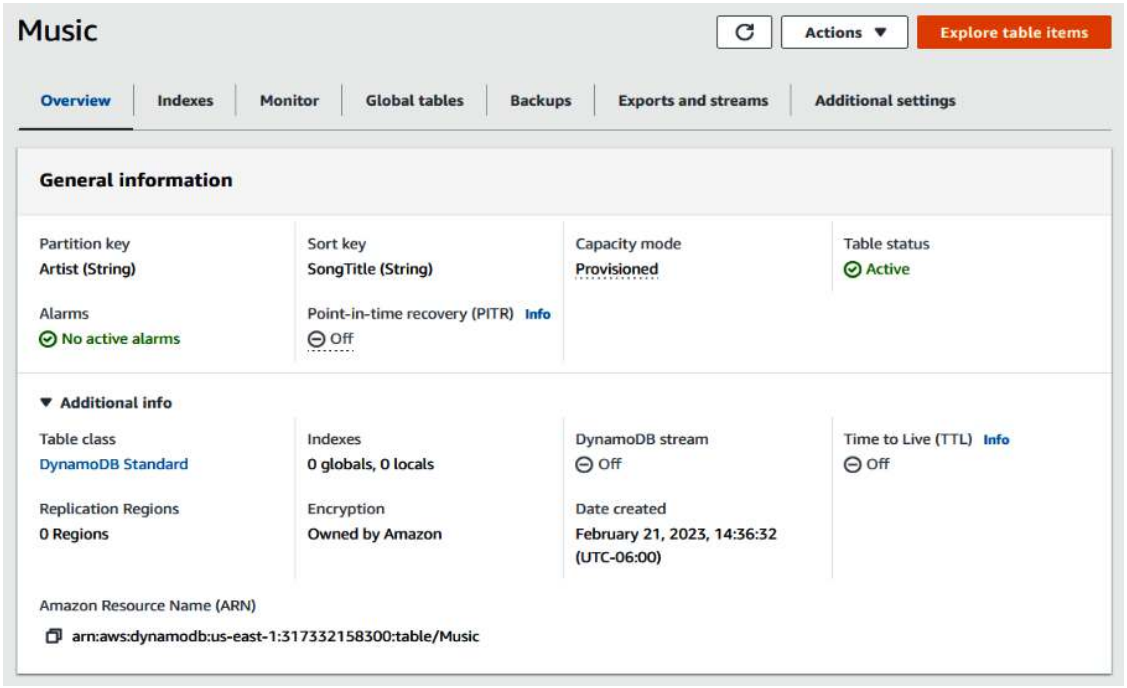


Figure 4.7 – DynamoDB "Music" table

Click **Explore table items**, and you can scan the table or run a query against the table. *Figure 4.8* shows the result of scanning the table:

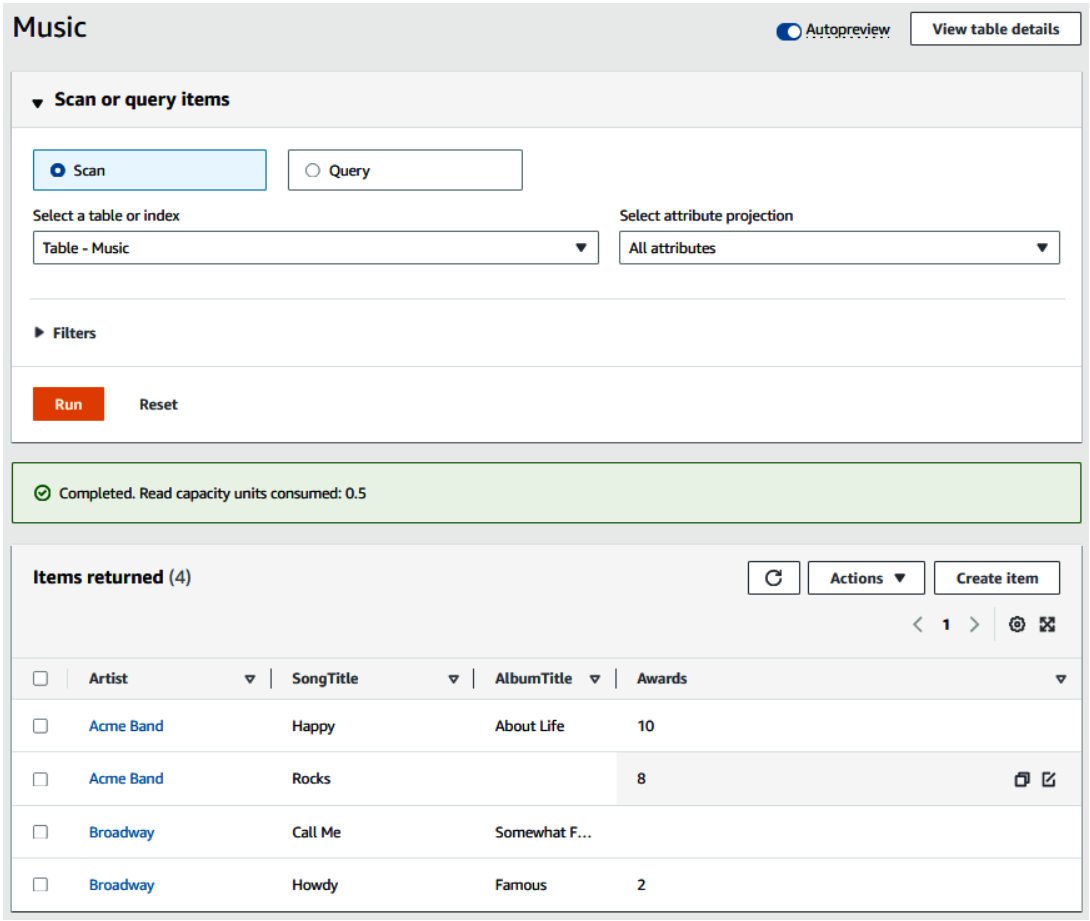


Figure 4.8 – Scan the "Music" table

Figure 4.9 shows the result of running a query by listing all the items for **Artist | Broadway**:

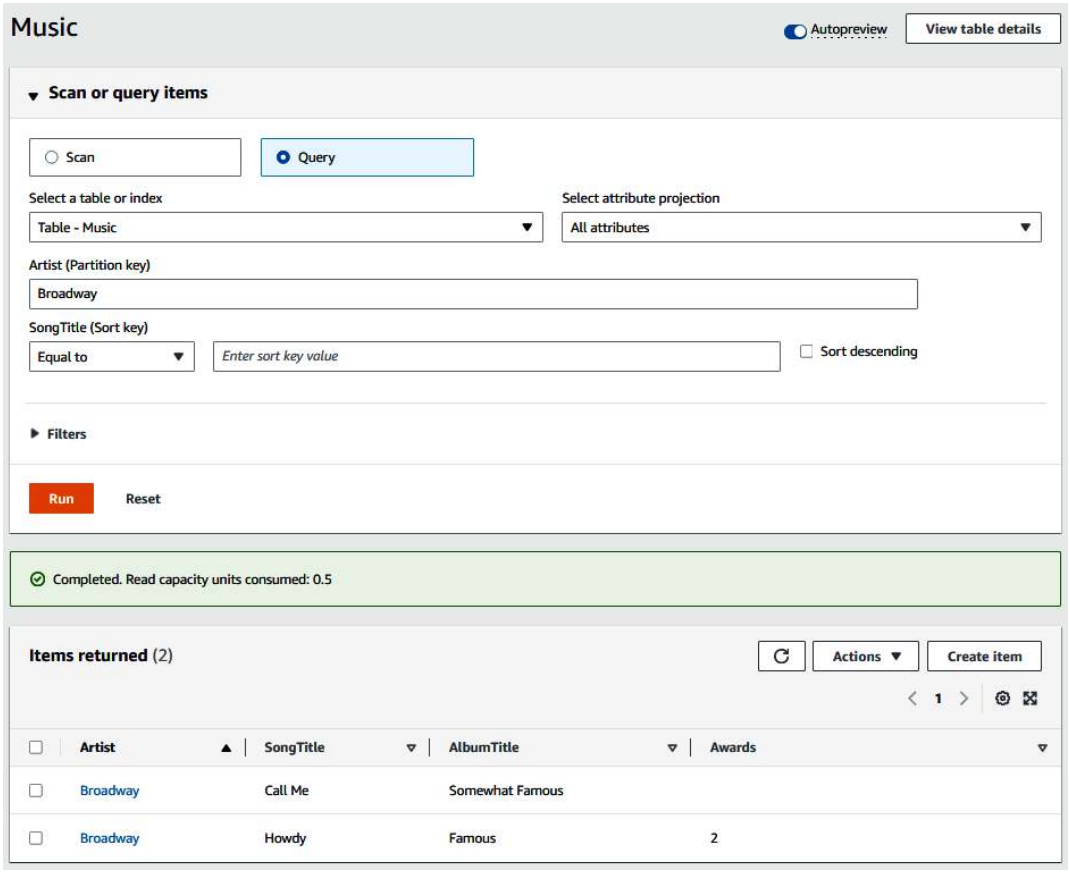


Figure 4.9 – Query the "Music" table

As we can see from *Figure 4.9*, DynamoDB provides a range of features for managing and querying data, including built-in support for indexing, filtering, and sorting, as well as a powerful query language called AWS Query and Scan. It also integrates with other AWS services, such as AWS Lambda and Amazon CloudWatch, for seamless integration with other parts of an AWS application stack. Overall, DynamoDB provides a scalable, flexible, and high-performance NoSQL database solution for a wide range of applications, from mobile and web applications to large-scale enterprise systems.

We have discussed Amazon RDS and DynamoDB. One of the issues with databases is performance, specifically when a table in the database is accessed by many users or applications and poses delays in responding to queries against it. One solution is caching the table into memory. This will lead us to the next section about Amazon ElastiCache.

Amazon ElastiCache

ElastiCache is a fully managed, in-memory data caching service provided by Amazon in the cloud. ElastiCache supports two popular in-memory data stores, Redis and Memcached, which can be used as caching layers for a wide range of applications, including web applications, mobile applications, and gaming applications.

ElastiCache provides automatic scaling and availability, with the ability to scale up or down based on the needs of the application. It also integrates with other AWS services, such as Amazon EC2, AWS Lambda, and Amazon RDS, for seamless integration with other parts of an AWS application stack. ElastiCache provides a range of features for managing and monitoring the caching environment, including automatic failover and automated backups.

ElastiCache also supports a range of use cases, including read-heavy workloads, session management, caching database queries, and real-time analytics. It also supports a range of data structures, including strings, hashes, sets, and lists, and provides a powerful query language for searching and manipulating data.

We will now create an ElastiCache instance and connect it to a client application:

1. **Create an EC2 instance with an SG**: This is to allow default SSH from the computer where you will access the ElastiCache cluster. Use the `VPC/subnet` for the EC2 instance, which we will name `EC2-1`, with an SG of allowing `SSH from my computer` so we can access the EC2 instance from our home computer. Save the key pair and the instance's public IP address. In our case, they are `kp.pem` and `3.238.218.212`.

   Create an SG named **RedisSG** to allow `SSH` from the previous SG.

2. **Go to the Amazon ElastiCache service**: Click **Get started**, then click **Create cluster**. Choose **Create Redis cluster**. Choose **RedisSG** as the security group, and select **minimum configuration**. *Figure 4.10* shows the details after the cluster has been created:

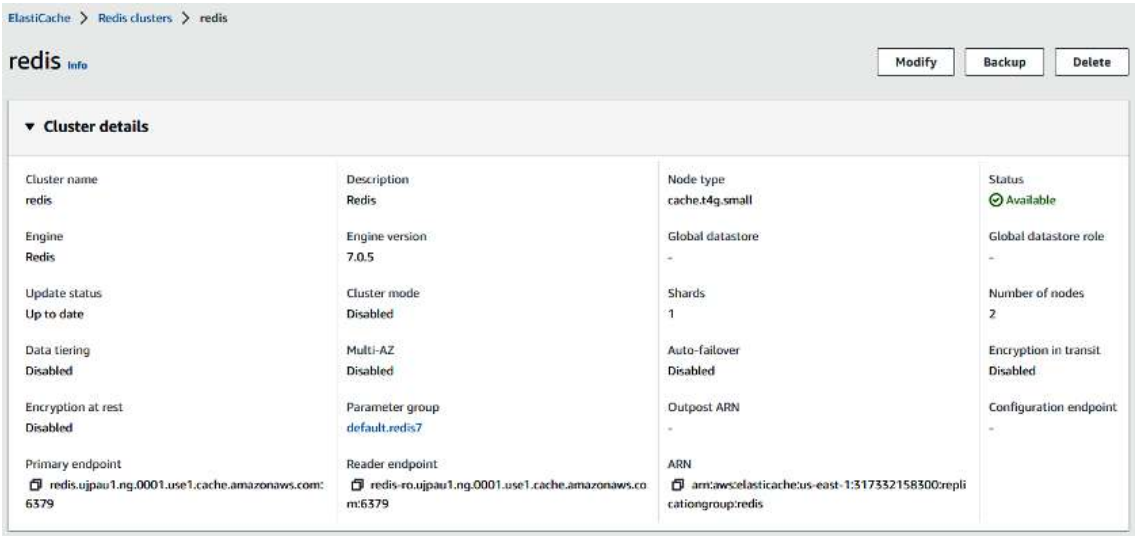


Figure 4.10 – Redis cluster details

3. **Access the Redis cluster from RedisInsight**: Just like the `PUTTY` tool we used to access Linux EC2 instances in the cloud, RedisInsight is a tool to access the Redis cluster remotely. After launching RedisInsight, click **Add Database**, then fill in the information for the cluster. Fill in **Host** using the primary endpoint copied from the Redis cluster details, leaving the default Redis port of `6379`. Since we will use the EC2 instance as the SSH tunnel, check **Use SSH Tunnel** and enter the instance's public IP address for **Host**, leaving the SSH port `22`. Check **Private Key** and copy the key from the `kp.pem` file as shown in *Figure 4.11*:

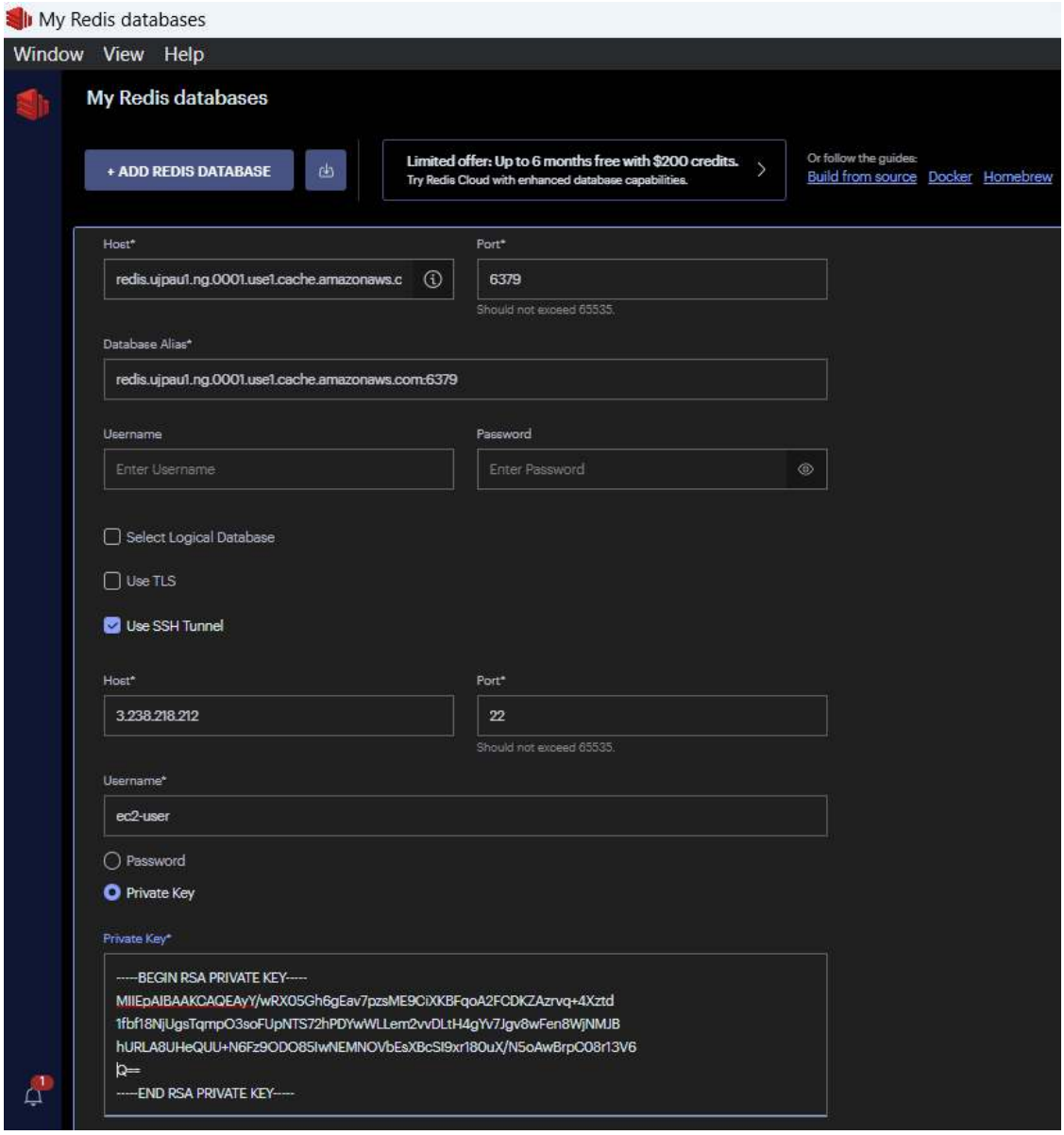


Figure 4.11 – Connecting to the Redis cluster

Once the Redis database is added, we can use **Browser**, **Workbench,** and the **Analysis** tools from the left side. More details are available at `https://redis.io/docs/ui/insight/`.

Amazon ElastiCache provides a highly scalable, fully managed in-memory data caching service that can improve application performance and reduce the need for expensive database queries. It is a popular choice for organizations of all sizes, from start-ups to enterprises that need to manage their caching environment in the cloud.

So far, we have learned about the Amazon cloud databases that support OLTP, including RDS, DynamoDB, and ElastiCache. There is another type of database that supports OLAP, which we refer to as data warehouses. Let's examine an AWS data warehouse: Redshift.

Amazon cloud data warehouse service

Amazon Redshift is a cloud-based data warehousing service that is designed to provide fast querying and analysis of large datasets. Redshift is a fully managed service where AWS handles the infrastructure and administration tasks, such as hardware provisioning, software installation, and backup and recovery, to allow users to focus on their data and analysis without worrying about the underlying infrastructure.

Redshift is based on a columnar storage format, which provides efficient storage and fast querying of large datasets. It can scale up or down as needed, with pay-as-you-go pricing based on the amount of data stored and the number of queries made. It also integrates with other AWS services, such as Amazon S3, Amazon EMR, and Amazon Kinesis, for seamless integration with other parts of an AWS application stack. Redshift supports standard SQL and BI tools such as Tableau, MicroStrategy, and Looker, allowing users to perform complex analytics and reporting on their data. It also provides a range of features for managing and optimizing queries, including query monitoring, query profiling, and automatic query optimization. Redshift is highly secure, with support for encryption at rest and in transit, as well as fine-grained access control through AWS IAM. It is also designed for high availability, with automatic replication of data to multi-AZs and automatic failover in the event of a node failure.

We will now create a Redshift instance and connect it to a client application:

1. **Create a security group**: This allows Redshift traffic from the computer where you will access the data warehouse, as shown in *Figure 4.12*:

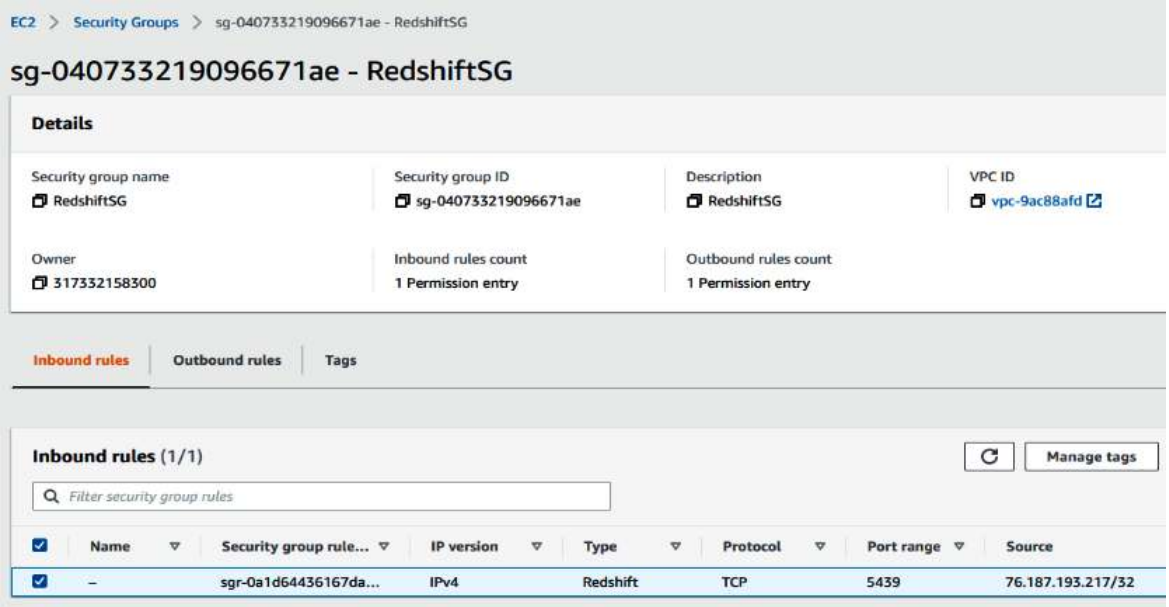


Figure 4.12 – SG for Redshift access

2. Create a subnet group that is designated for the Redshift clusters running in the VPC environment. Create an IAM role that has permissions to copy, unload, and query data with Amazon Redshift, and to run SELECT statements for related services, such as Amazon S3, Amazon CloudWatch logs, and so on.
3. Navigate through **AWS Console | Amazon Redshift | Create Cluster**: Fill in the cluster configurations, including name, SG, subnet group, and role and choose the minimum requirements in monitoring, backup, and so on. Make sure the cluster is publicly available. When the cluster is created, copy its endpoint:

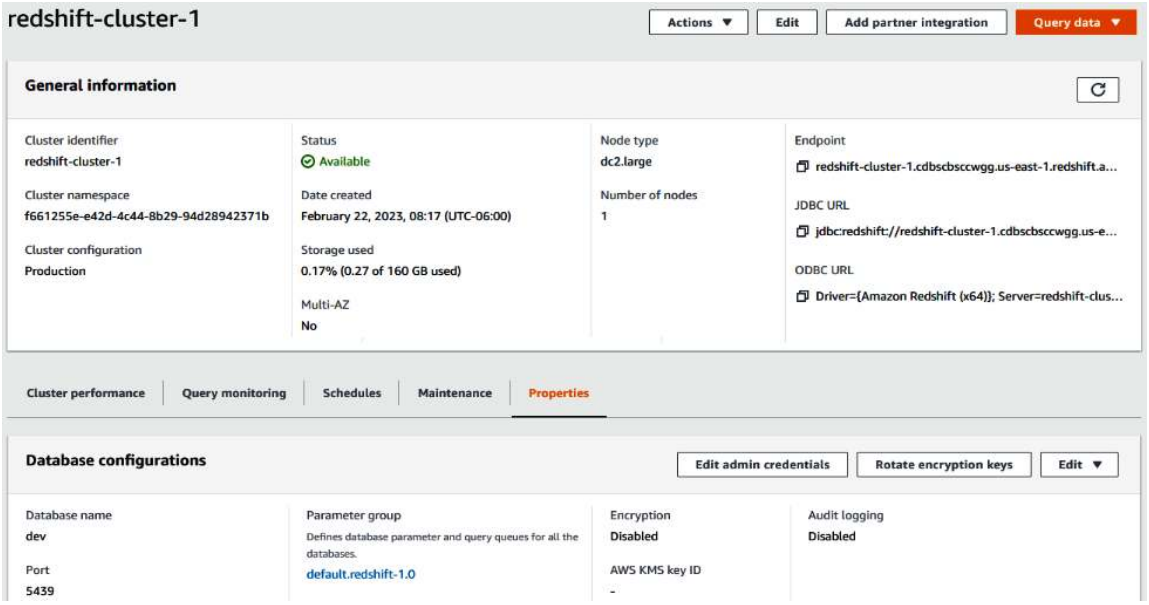


Figure 4.13 – Redshift cluster

4. **Access the Redshift database from DBeaver**: Just like the `PUTTY` tool we used to access Linux EC2 instances in the cloud, **Dbeaver** is a tool to access cloud databases remotely.

   Open **DBeaver | Database | New Database Connection**. In the new pop-up window, choose **Redshift** as the database type, and fill in the endpoint copied from the AWS Redshift dashboard, the **Database** name, **Username**, and **Password**, as shown in *Figure 4.14*:

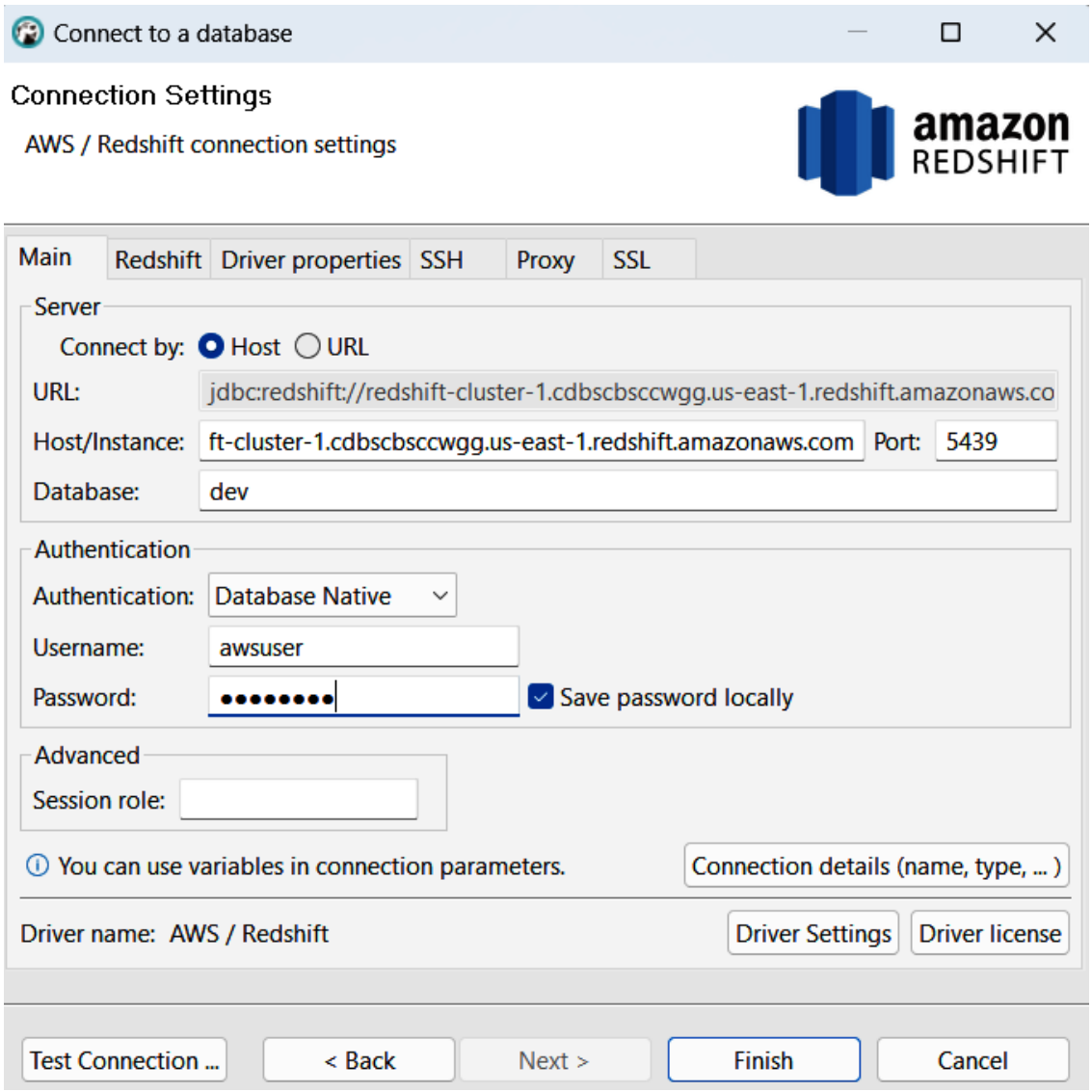


Figure 4.14 – Connect to Redshift cluster

Figure 4.15 shows the connection to the Redshift cluster. You can see the database schemas and use SQL Editor to compile and run queries again in the data warehouse:

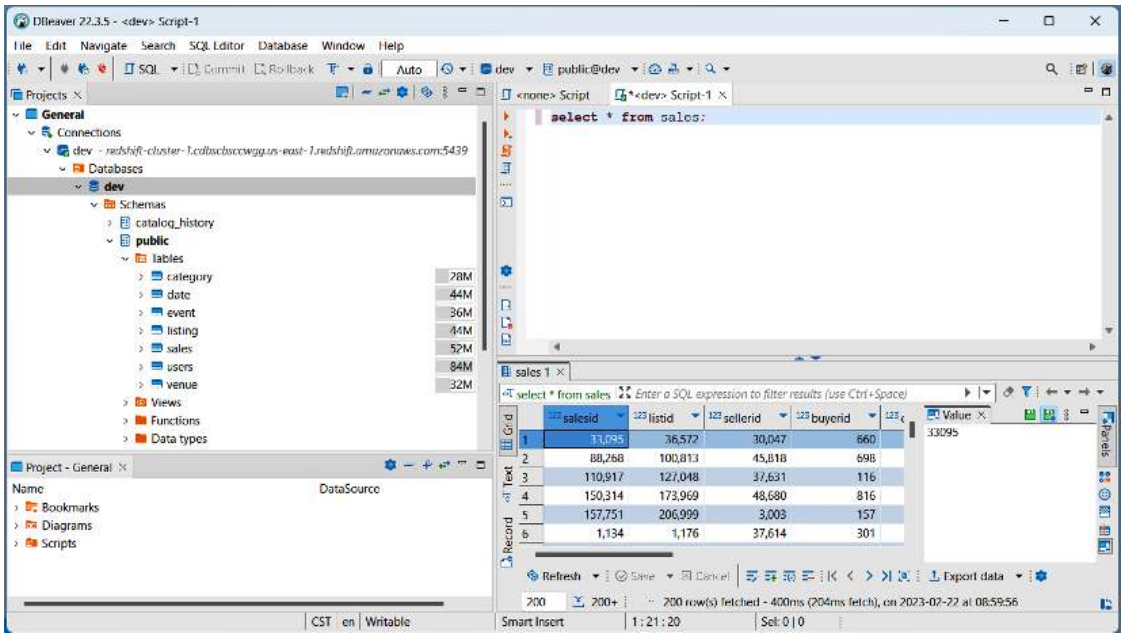


Figure 4.15 – Connection to Redshift cluster

Redshift provides a scalable, flexible, and high-performance data warehousing solution for a wide range of applications, from small data marts to large-scale enterprise data warehouses. More details are available at `https://docs.aws.amazon.com/redshift/`.

Summary

In this chapter, we discussed Amazon cloud databases including RDS, Aurora, DynamoDB, ElastiCache, and Redshift. We provisioned simple database instances and showed how to connect them and use the services. With all the different data schemas, business use cases, and application requirements such as size, scale, performance, and cost, selecting the right database is a critical step. More details are included at `https://aws.amazon.com/startups/start-building/how-to-choose-a-database/`.

In the next chapter, we will further explore Amazon's big data services.

Practice questions

Questions 1-10 are based on *Figure 4.16*:

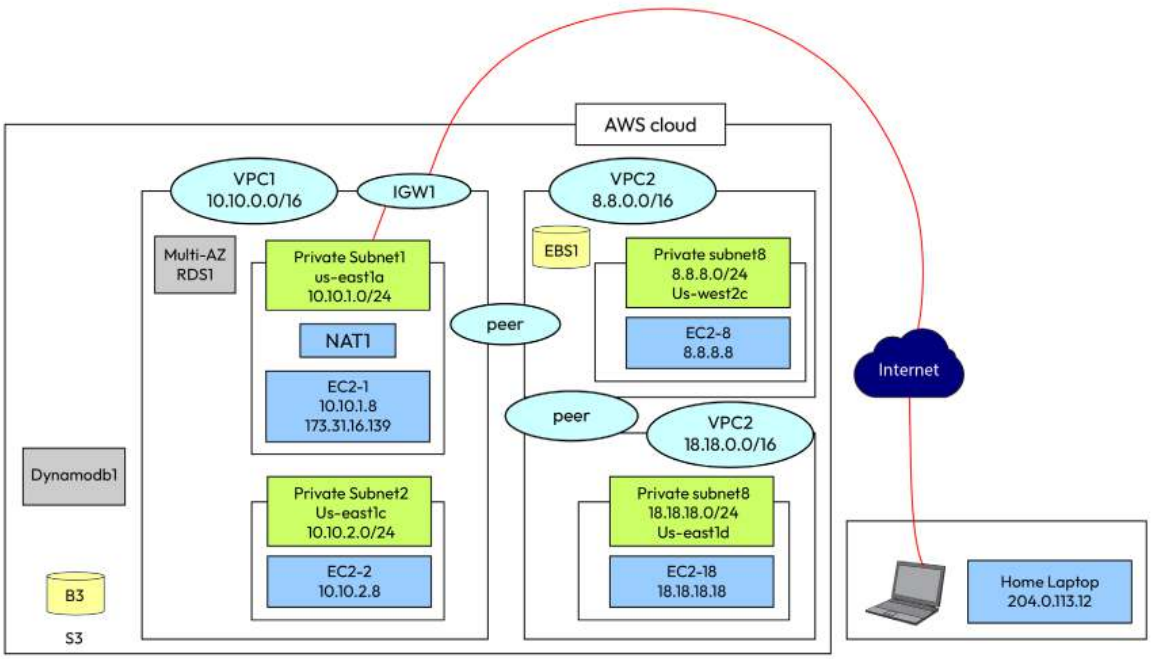


Figure 4.16 – Cloud architect diagram

1. RDS1 is created in VPC1. How would you configure it to be accessible from your home laptop?

   A. Configure SG for RDS1 and NACL for subnet1/subnet2.
   B. Configure SG for RDS1.
   C. Configure NACL for VPC1.
   D. Configure SG for VPC1.

2. Which of the following is not true?

   A. EC2-18 can access RDS1 via VPC peering.
   B. EC2-2 can access RDS1 within VPC1.
   C. EC2-8 can access RDS1 via VPC peering.
   D. EC2-1 can access RDS1 within VPC1.

3. An engineer is developing a web server W1 on EC2-18. W1 needs to access Dynamodb1. What is your recommendation?

   A. Create an EC2 role R1 for Dynamodb1 access, assign R1 to EC2-18, and create an endpoint for Dynamodb1 in VPC3.

   B. Create an EC2 role R1 for Dynamodb1 access, and assign R1 to EC2-18.

   C. Create an endpoint for Dynamodb1 in VPC3.

   D. Create an EC2 role R1 for Dynamodb1 access, assign R1 to EC2-18, create an endpoint for Dynamodb1 in VPC1, and create peering between VPC1 and VPC3.

4. RDS1 was created without encryption, but now encryption is required for compliance reasons. What is your recommendation?

   A. Back up RDS1 and enable encryption on the fly.

   B. Export RDS1. Create a new RDS2 with encryption and import from the RDS1 export.

   C. Snapshot RDS1. Create a new RDS2 and restore it from the RDS1 snapshot.

   D. Snapshot RDS1. Create an encrypted copy of the snapshot. Create a new RDS2 from the encrypted snapshot.

5. RDS1 is not configured with auto-scaling, and it ran out of storage space in the last hour. What is your recommendation?

   A. Add more space to RDS1 using the `ModifyDBInstance` option. Then, enable storage auto-scaling thereafter.

   B. Create a new RDS2 with more storage space. Import it from the latest backup.

   C. Enable storage auto-scaling for RDS1 immediately.

   D. Configure a new read replica of RDS1 with more storage space.

6. The master database for RDS1 is in `Subnet1`. Where shall the slave database be placed for RDS1?

   A. Subnet2

   B. Subnet8

   C. Subnet1

   D. VPC2

7. RDS1 has performance issues with table1, which is updated by many users. What's your recommendation?

   A. Use Elastic Cache.

   B. Use Read Replica for RDS1.

C. Create RDS2.

D. Separate table1 into multiple tables.

8. EC2-8 needs to access Dynamodb1 without trespassing the internet. What needs to be done?

A. Create a DynamoDB endpoint in VPC1 and access Dynamodb through VPC-peering.

B. Create a DynamoDB endpoint in VPC2 directly.

C. Create a DynamoDB endpoint in subnet2.

D. Create a DynamoDB endpoint in subnet8.

9. EC2-2 needs to add a new EBS volume. Where shall the new volume be created?

A. US-EAST1a

B. US-EAST1b

C. US-EAST1d

D. US-WEST1c

10. EC2-2 has a web application that needs to generate new reports from RDS1. Without impacting the database, what's your suggestion?

A. Create a read replica and allow the application to access the replica.

B. Upgrade RDS1 to Aurora.

C. Copy RDS1 to RDS2. Provide EC2-2 access to RDS2.

D. Load RDS1 to ElastiCache, and allow the application to access the cache.

Answers to the practice questions

1. A
2. A
3. A
4. D
5. A
6. A
7. A
8. A
9. D
10. A

Further reading

For further insights into what you've learned in this chapter, refer to the following links:

- `https://aws.amazon.com/products/databases/`
- `https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html`
- `https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html`
- `https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_GettingStartedAurora.html`
- `https://docs.aws.amazon.com/elasticache/`